

UNIVERSITÀ DI PISA

DATA MINING 1 25/26 PROJECT REPORT

Master's Degree:

Data Science & Business Informatics

Pietro Gagliardi

Riccardo Ranieri

Giambattista Bernardo

Professors:

Dino Pedreschi

Riccardo Guidotti

Contents

1 Introduction **2**

2 Data Understanding and Preparation **2**

2.1 Data Semantics 2

2.2 Place holder values and categorical encoding 2

2.3 Distributional analysis 3

2.4 Attribute manipulation 6

3 Clustering **7**

3.1 KMeans 7

3.2 DB-scan 9

3.3 Hierarchical clustering 10

3.4 Final discussion 11

4 Classification **12**

4.1 KNN 13

4.2 Naive Bayes 13

4.3 Decision Tree 14

4.4 Final discussion 16

5 Regression **17**

5.1 Univariate Regression 17

5.2 Multivariate Regression 17

6 Pattern mining **18**

6.1 Association rules 20

1 Introduction

The goal of this data mining project is to analyze a board game dataset in order to understand its main characteristics and extract meaningful insights.
To achieve this objective, several data mining techniques have been applied. During the data understanding and preparation phase, the dataset was cleaned and pre-processed to ensure its suitability for analysis. Subsequently, a range of clustering methods was used to identify natural groupings within the data. Classification and regression algorithms were then applied to both categorical and numerical variables to predict selected target values. Finally, pattern mining techniques were employed to uncover significant relationships among the data.

The insights obtained from this analysis provide a deeper understanding of the structure and behavior of the board game dataset and may support data-driven decision making.

2 Data Understanding and Preparation

2.1 Data Semantics

The board game dataset, sourced from the website BoardGameGeek, contains a wide range of information about table-top games. The dataset includes attributes describing various aspects of each game, such as its category, recommended age, difficulty level, and many other characteristics; all the variables are summarized in Table 1.

Table 1: Dataset variable groups

Group	**Variables**	**Type**
Identifiers	BGGId, Name	int64, object
Game information	Description, YearPublished, ImagePath, Family	object / int64
Players	MinPlayers, MaxPlayers, BestPlayers, GoodPlayers	int64 / object
Ratings and complexity	GameWeight, ComWeight, Rating	float64 / object
Community statistics	NumOwned, NumWant, NumWish, NumUserRatings, NumComments, NumWeightVotes	int64
Playtime and age	MfgPlayTime, ComMinPlaytime, ComMaxPlaytime, MfgAgeRec, ComAgeRec	int64 / float64
Game relations	NumExpansions, NumAlternates, NumImplementations, IsReimplementation	int64 / bool
Crowdfunding	Kickstarted	bool
Game ranks	Rank:StrategyGames, Rank:Abstracts, Rank:FamilyGames, Rank:Thematic, Rank:CGS, Rank:Wargames, Rank:PartyGames, Rank:ChildrensGames	int64
Game categories	Cat:Thematic, Cat:Strategy, Cat:War, Cat:Family, Cat:CGS, Cat:Abstract, Cat:Party, Cat:Childrens	bool (0/1)

2.2 Place holder values and categorical encoding

Only four attributes in the dataset contain explicit missing values; the sparsity is significant only for LanguageEase (n=5891) and ComAgeRec (n=5530), while ImagePath and Description exhibit negligible missingness (17, 1).
All the **"Rank:"** attributes (one for each category) contained, for the most part, the sentinel value **21926**, which is used to identify board games that do not belong to the corresponding category. In

addition, placeholder values were also identified in the following attributes: **BestPlayer**, **GoodPlayers**, **ComMaxPlaytime**, **ComMinPlaytime**, and **MfgPlaytime, GameWeight and ComWeight**. In particular, identifying placeholders within the 'GameWeight' and 'ComWeight' variables,both measures of perceived complexity, required a more nuanced analysis. Although these metrics are defined on a 0 to 5 scale, an inspection of their distribution histograms reveals a complete absence of values between 0 and 1. This evidence suggests that the functional rating scale ranges from 1 to 5, while the value 0 serves as a sentinel representing missing or undefined data.

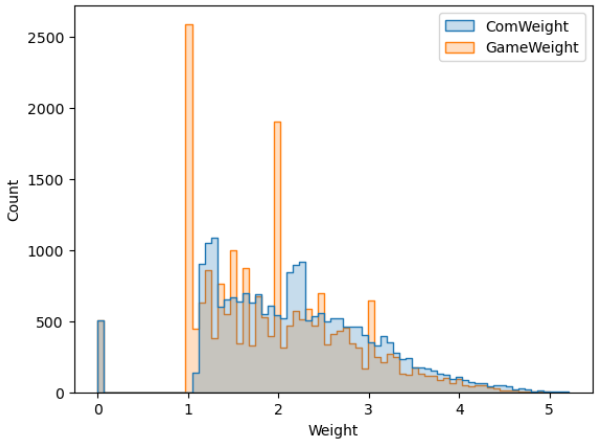
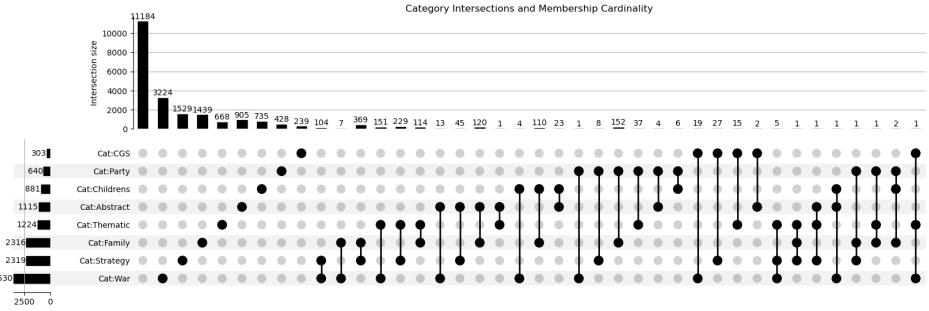


Figure 1: distribution of GameWeight and ComWeight

All the **"Cat:"** attributes (8 in total) were encoded using a multi-hot representation to indicate category membership. Further analysis shows that almost half of the games (11,184) are not associated with any category, while only a small number of games (13) belong to three different categories simultaneously.



0	1	count
8	0	11184
7	1	9167
6	2	1561
5	3	13

Figure 2: Count of the games by category [library: upset plot]

2.3 Distributional analysis

A preliminary analysis on the distribution of the numerical attributes show a strong presence of skewness in the data, especially right skewness with long tail.

Such skewness is not incidental but reflects intrinsic properties of the domain, where user engagement and popularity-related metrics naturally follow heavy-tailed distributions.

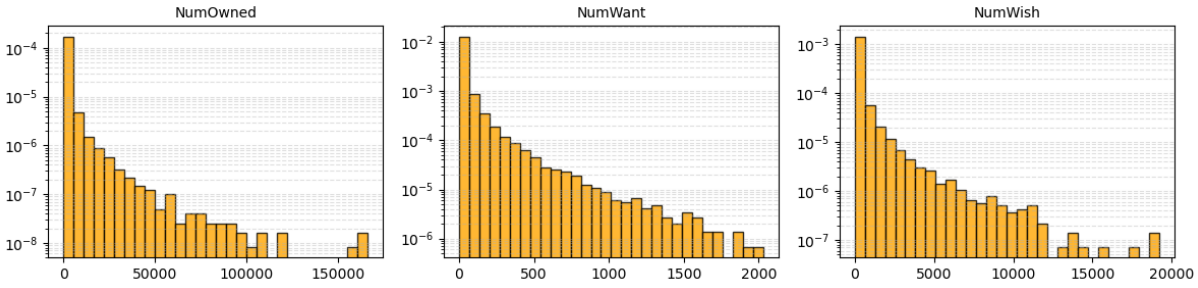


Figure 3: Distribution of Popularity measures (log scale)

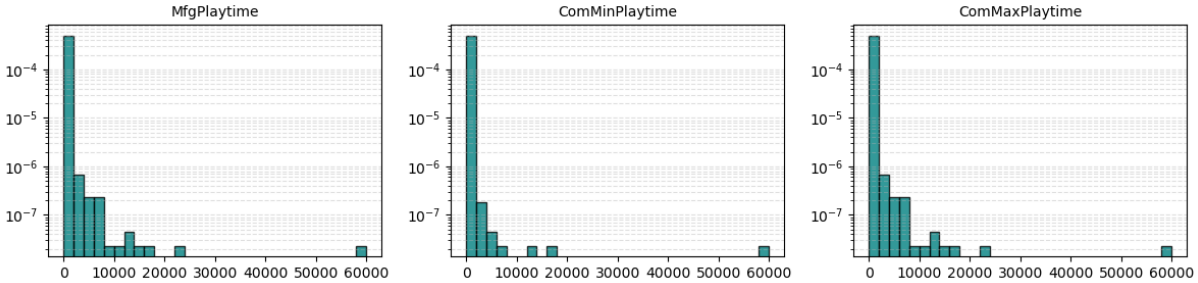


Figure 4: Distribution of Playtime measures (log scale)

The marked long-tailed structure observed in the distributions indicates a significant presence of extreme values. An in-depth analysis confirms that outliers are widespread across almost all numerical attribute. For many attributes, the presence of outliers follows a systematic pattern that is intrinsically linked to the feature's domain logic rather than being the result of measurement errors; for instance in attributes like the playtime one or NumWish it's understandable to have board games that accumulates very high values.

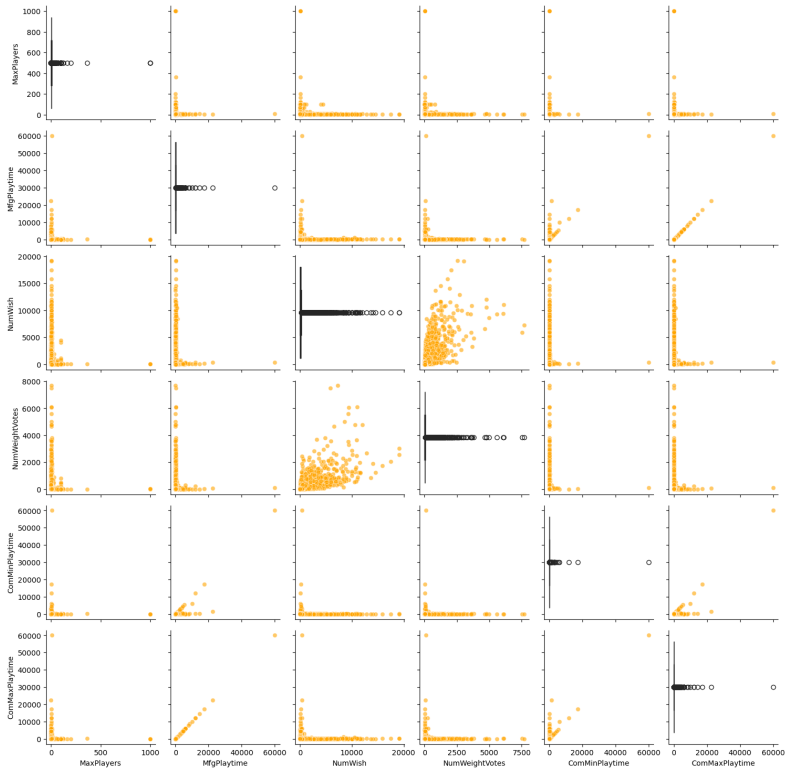


Figure 5: Scatter plots highlighting the presence of outliers with a box plot in the diagonal

Finally, a correlation matrix was computed to investigate the relationships among the different attributes.

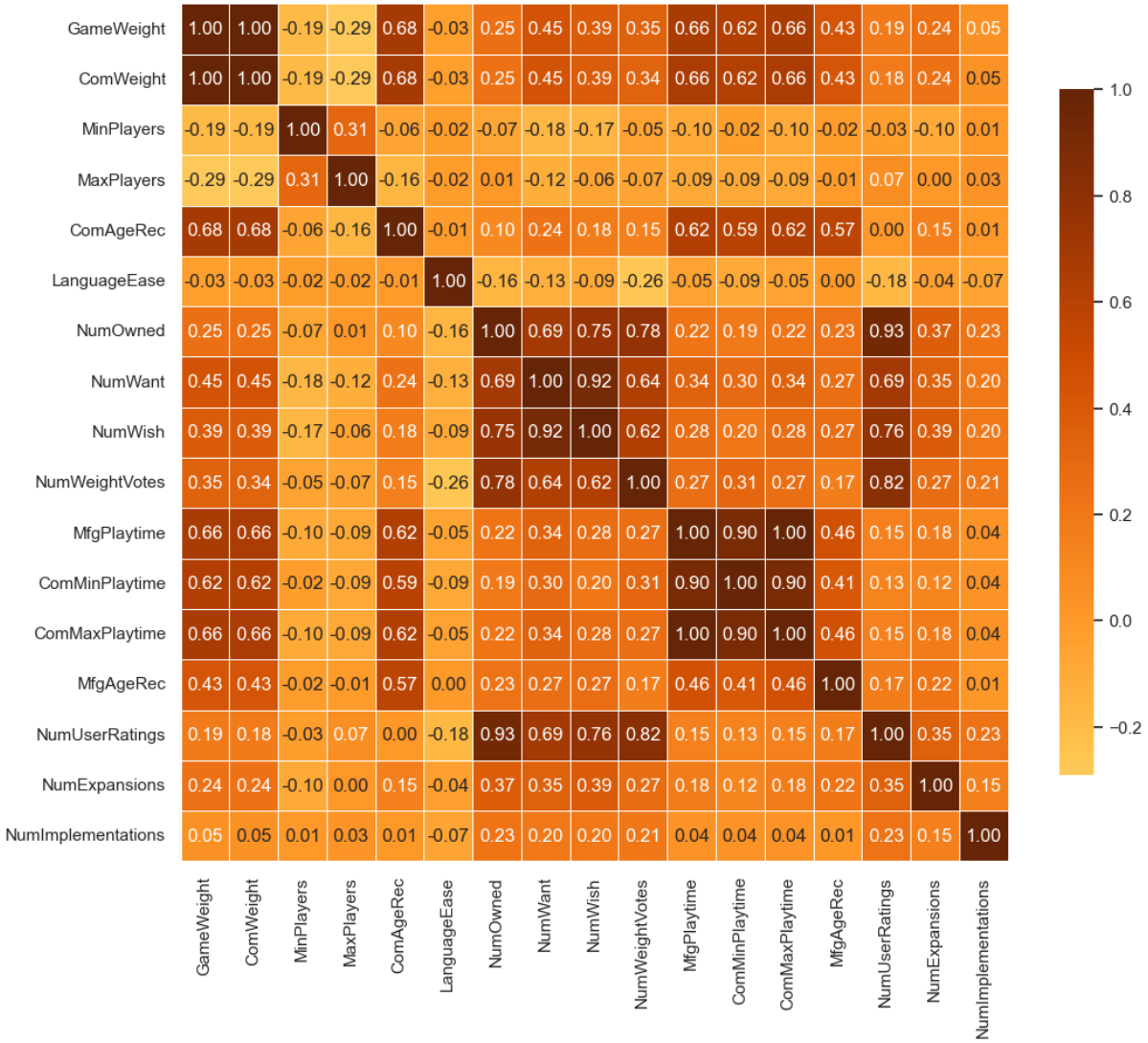


Figure 6: Correlation Matrix of Selected Numeric Features (Spearman correlation)

As shown in Figure 6, a few pairs of features exhibit a strong positive correlation, while negative correlations are almost absent. This suggests that some variables tend to increase together, potentially indicating redundancy or overlapping information among features, most feature pairs show a low correlation values.

The observations done in this part of the report will provide the primary motivation for the subsequent data preparation phase.

2.4 Attribute manipulation

The following attributes were removed from the dataset:

Table 2: Dropped variables and motivation

Dropped Variable	**Motivation**
Description, ImagePath	Considered not relevant for the purpose of the analysis.
GoodPlayers, BestPlayers	More than 92% of missing values, making the variables unreliable for analysis.
NumComments	The column contains only zero values, thus providing no discriminative information.
NumUserRating	has a correlation of 0.93 with NumOwn implicating a strong redundancy
GameWeight	has a correlation of 1 with ComWeight
NumWant	has a high correlation with NumWish (0.92) and gives the same informations
MfgPlaytime	has a correlation of 1 with ComMaxPlaytime

All rows (n = 406) with **YearPublished** < 1950 were removed from the dataset. This preprocessing step mitigated the presence of extreme outliers without significantly reducing the size of the dataset.
To improve data quality, the placeholder value 21,926 in the **Rank** columns was encoded as a 'Missing Value' (NaN), ensuring it does not interfere with further analysis; However, after further evaluation, the Rank features were excluded from the final dataset, as they were not considered informative for the analysis task.
The attribute **NumAlternates** was discretized into five categories, transforming it from a continuous variable into a categorical one. {'Nessuno (0)': 12316, 'Uno (1)': 3696, 'Pochi (2-5)': 3442, 'Moderati (6-10)': 964, 'Molti (11+)': 355}, This was done in order to reduce the effect of outliers and to obtain a more suitable attribute for classification purposes. The categorical attribute**Rating** had its 3 categories transformed:

$$Low- > 1|Medium- > 2|High- > 3$$

Encoding it as an ordinal numerical feature was done to improve the performance of the classifiers by providing a more informative distance measure.
The attribute **Family**, which contains the name of the family to which each game belongs (e.g., a game named "Monopoly Canada Edition" has family "Monopoly"), was converted into a boolean attribute indicating whether the game belongs to a family (1) or not (0). The column was also renamed to **BelongToFamily**. This transformation allows the feature to be utilized in the analysis, even though it entails some loss of detailed information about the family name.
The attribute **ComAgeRec** was converted from `float64` to `int64`. The following attributes—**LanguageEase**, **NumOwned**, **NumWish**, **NumWeightVotes**, **NumImplementations**, and **NumExpansions**—were transformed using a logarithmic transformation in order to reduce noise and mitigate the impact of outliers.
All rows where ComMinPlaytime > ComMaxPlaytime were eliminated from the dataset.

3 Clustering

The dataset has been analyzed with three types of clustering algorithms: center-based, density based and hierarchical. For this task, only a few features have been considered.

3.1 KMeans

First step after standardized the dataset using the *MinMaxScaler*, was to select the best value of the hyperparameter k, which it's been calculated through both Sum of Squared Errors (SSE) and Silhouette Score in a range [2, 50] and plotted their values as seen below.

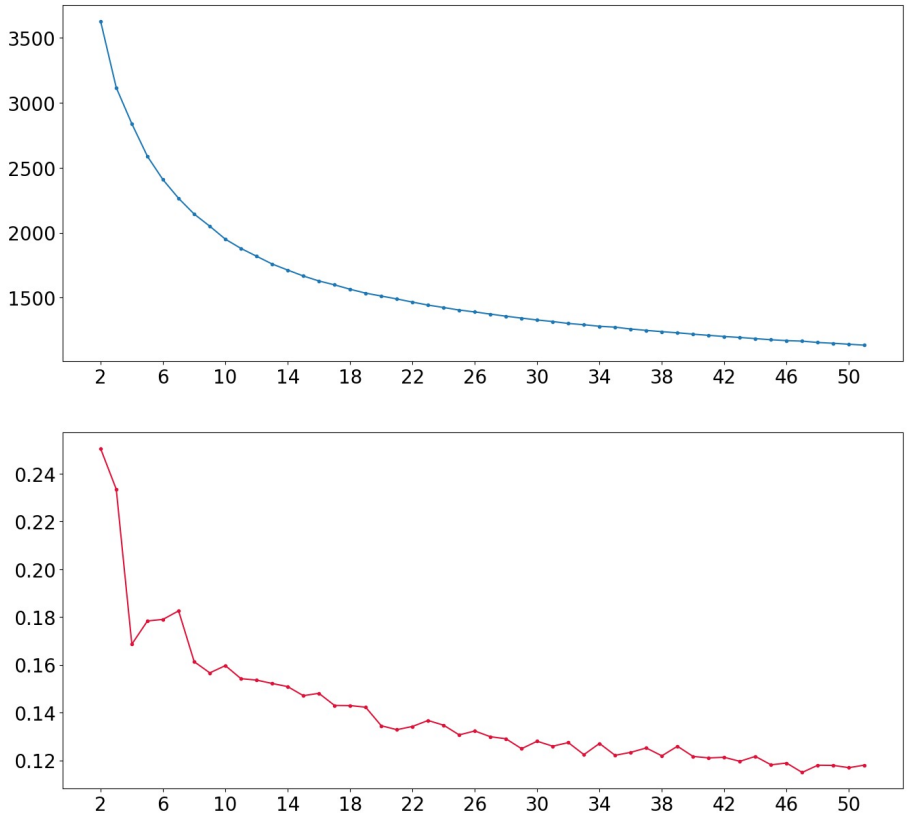


Figure 7: Elbow Method and Silhouette Score analysis

Based on the trend of the two curves, the best hyperparameter selected for executing KMeans was $k = 6$, representing the optimal trade-off between the metrics.

The choice of $K = 6$ is further justified by comparing it with $K = 7$. Although increasing K to 7 yields a marginal improvement in the Silhouette score (0.185 vs 0.179), it results in a suboptimal distribution characterized by the emergence of a micro-cluster containing only 1.03% of the data (215 items). This suggests that $K = 7$ begins to isolate statistical outliers rather than identifying a meaningful market segment. Therefore, $K = 6$ represents the optimal trade-off between mathematical cohesion and structural balance.

Cluster	N	%
Cluster 0	2672	12.86
Cluster 1	5604	26.98
Cluster 2	4963	23.89
Cluster 3	2948	14.19
Cluster 4	2269	10.92
Cluster 5	2317	11.15
	Silhouette: 0.179	
	SSE: 2408.30	

Table 3: Cluster distribution

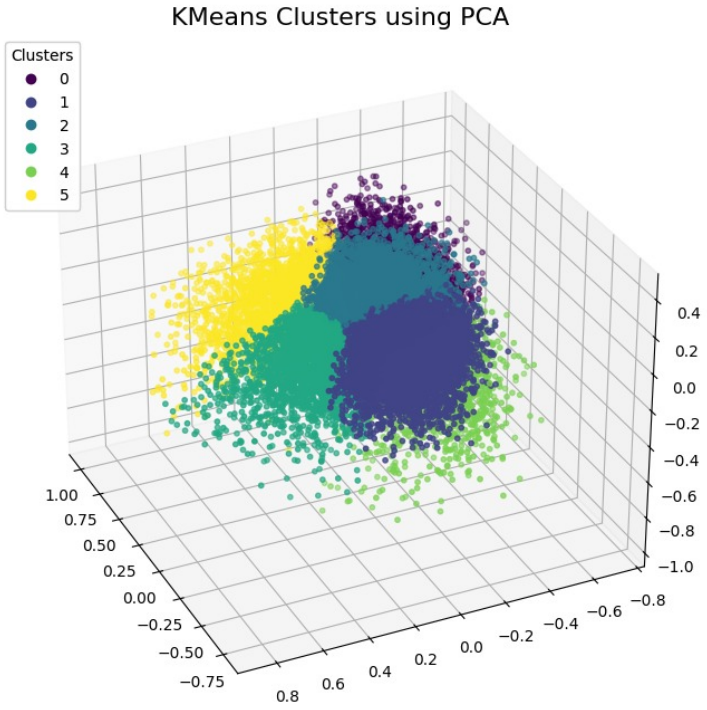


Figure 8: PCA visualization

From the table of cluster distribution, it can be seen that Clusters 1 and 2 account for approximately 51% of the total data points, representing the most common board game types. The remaining four clusters exhibit a remarkably balanced distribution, with shares ranging between 11% and 14%, ensuring that each segment represents a significant portion of the market.

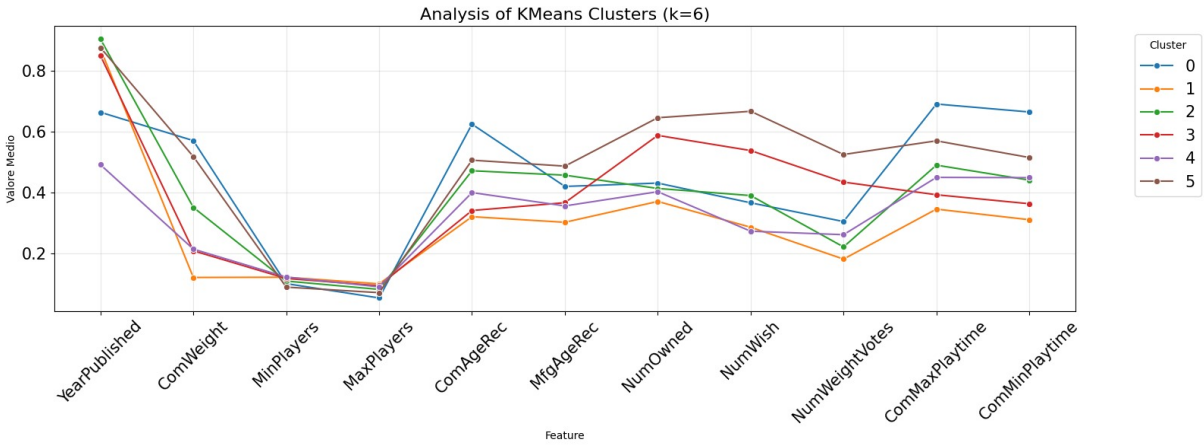


Figure 9: Snake Plot of centroids analysis

By looking at the Figure 9, visualizing the vertical spread between the lines and highlighting the "distance" between groups, it can be seen that the clusters tend to separate points mostly based on **ComWeight**, **ComAgeRec**, **NumWish**, and **ComMinPlaytime**. **YearPublished** acts as a highly discriminative feature in the clustering process, effectively separating the centroids based on the games' chronological age (vintage).

3.2 DB-scan

DB scan is a density based algorithm that works really well with noise and clusters with different shapes. Given the set of features (the same used for the other clustering algorithms), the optimal values for **epsilon** and **min_samples** were determined through a grid search over the following ranges: epsilon : [0.5, 1.0, 1.5, 2.0, 2.5, 3.0] and min_samples : [5, 10, 15, 20, 25, 30, 35, 40, 45, 50].

As shown in the figure, the best parameter combination in terms of silhouette score was obtained with **min_samples = 45** and **epsilon = 2.5**, The elbow method for DB scan confirmed that the best epsilon range is between 2 and 2.5 .

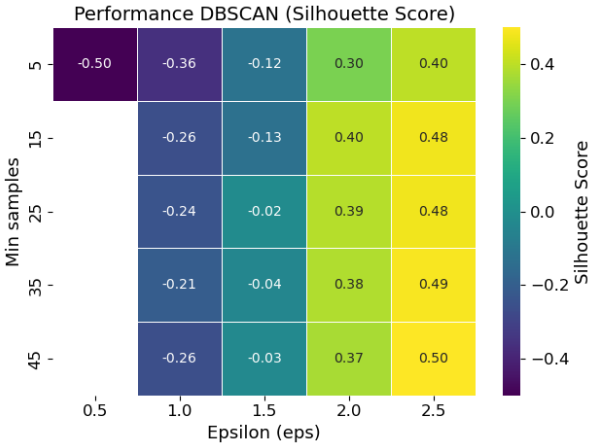


Figure 10: grid search for parameter tuning

However, these optimal parameters do not necessarily imply the discovery of a meaningful clustering structure. In several configurations, DBSCAN identified only a single cluster; this occurs because the Silhouette Score treats the noise class (label= -1) as a legitimate cluster, leading to artificially high scores when outliers are distant from the main data mass. The most balanced configuration yielding more than one cluster was found at epsilon = 1.5 and min_samples=40. This setup identified two clusters with a Silhouette Score of 0.12, which remains significantly low.

With a noise rate of 29.13%, it is evident that the dataset lacks well-defined, high-density regions. The underlying data distribution appears to be relatively continuous rather than organized into clearly separated dense manifolds. Consequently, the lack of distinct density 'valleys' between groups limits the effectiveness of a density-based approach like DBSCAN on this specific dataset.

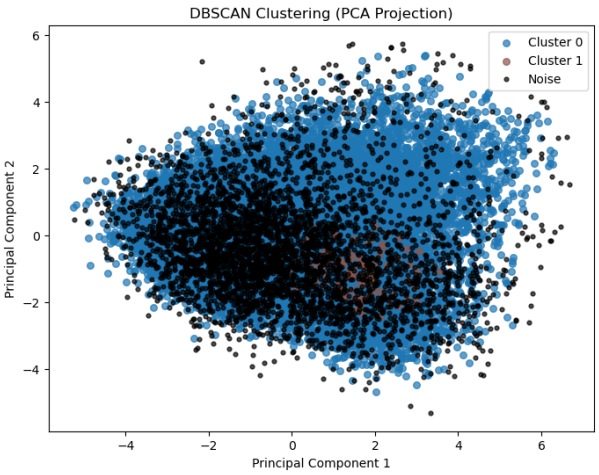


Figure 11: PCA of Db scan

This PCA visualization of the clustering shows how the data points form a single, continuous distribution without clear density gaps, confirming why the algorithm struggles to define robust cluster boundaries. The significant overlap between the identified groups justifies the low silhouette score and indicates that the underlying feature space behaves like a single manifold rather than a set of discrete categories. Ultimately, this projection highlights that density-based separation is limited in this dataset by the lack of isolated high-density regions.

3.3 Hierarchical clustering

In order to compare the results of the hierarchical clustering with the results of density-based and k-means we used the same features and the same k used in the k-means, despite the fact these features and k = 6 didn't give us the best solution possible with hierarchical. Running a hierarchical clustering on the entire data set would have been computationally expensive and the dendrogram would have been unreadable, so we decided to run a k-means with k = 2000 to reduce the dimensionality, then we took the indexes of the points of the data set closest to the centroids and used these points to build the dendrogram, for which we used the **Ward** method, because we obtained poor quality clusters with **average**, **single** and **complete** linkage.

We cut the dendrogram in order to obtain 6 clusters to compare it with k-means. A mapping function was then applied to propagate the hierarchical cluster labels from k-means micro-clusters to all individual data points, so all points belonging to the same micro-cluster were assigned to the same hierarchical macro-cluster. The silhouette score obtained was pretty low, sitting at 0.1325, this suggests a complex and overlapping structure of the clusters and a worse result than k-means. The only clusters that are better built are the cluster 3, we can see in the heatmap that it has complex and long duration games, and cluster 1, in which there are the more recent games due to the high weight of the **YearPublished**. In cluster 5 there are the games with largest number of players, but it has a serious problems of dimensions (only 248 data). So we can state that the hierarchical clustering was the worst algorithm to use with these features and this k.

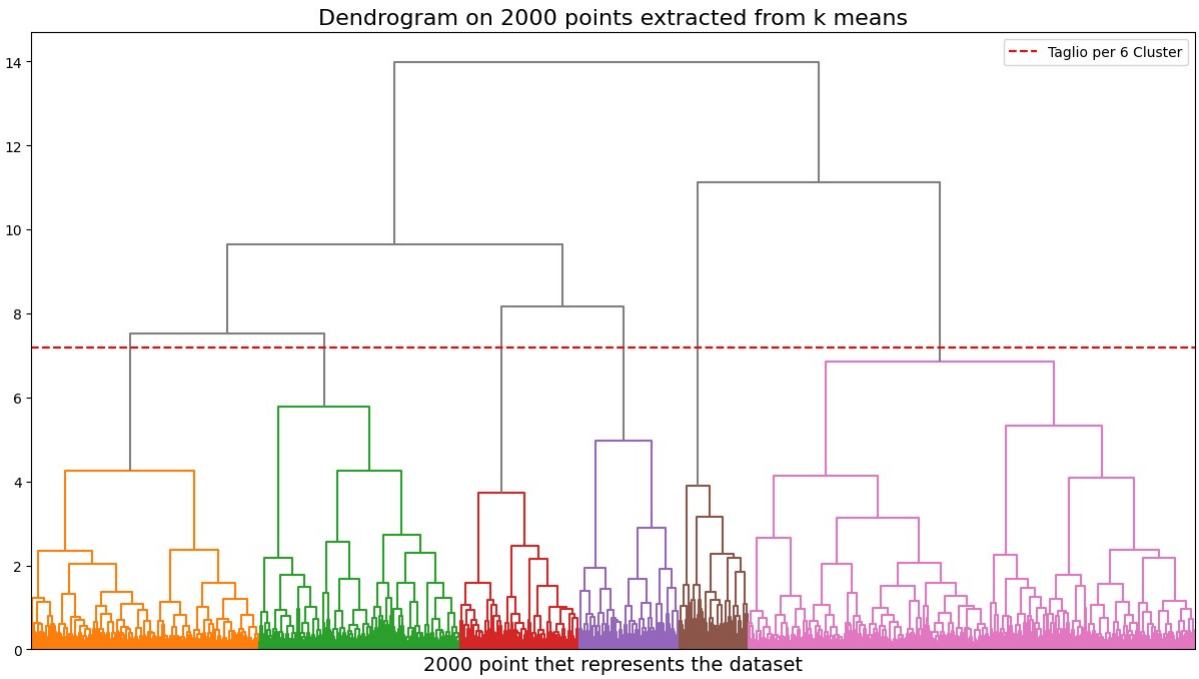
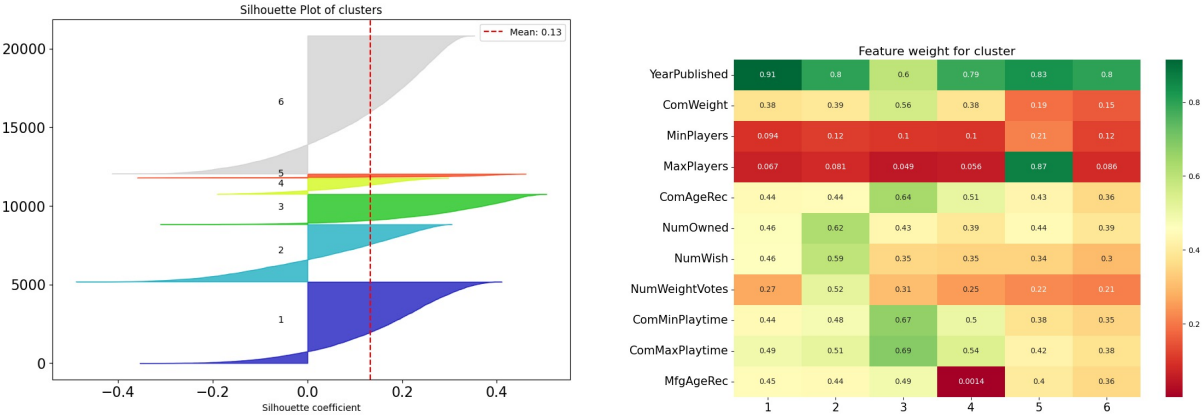


Figure 12: Dendrogram on 2000 points extracted from k-means.



(a) Silhouette Plot for single points.

(b) Heatmap of the weights of the feature.

Figure 13: Analysis of the quality of the clusters

3.4 Final discussion

Taking into account the results obtained by the three algorithms, it can be concluded that DBscan and hierarchical weren't able to create balanced clusters and instead created complex and overlapping structure. K-Means emerged as the optimal algorithm, yielding the best results, in terms of Silhouette Score and cluster distribution stability. The interpretative analysis brings out six distinct profiles:

- **Cluster 1**: consisting of a low difficulty, slightly higher average number of players and recommended age around 8 years old, it represents the most accessible games, designed for families or parties.
- **Cluster 2**: average difficult of 2.41 and **YearPublished** around 2014, represent recent games, more difficult and designed for a recommended age of $\tilde{1}$0-11 years old, higher than cluster 1, and longer games.
- **Cluster 3**: consist of a lower **ComWeight** (1.84), but a much higher **NumOwned** and **NumWish**.

These are more established games.

- **Cluster 4**: stand out for a lower **YearPublished** and a medium-low difficulty, it remarks old games, owned by an average part of the market but 'desired' by few. (2.69)
- **Cluster 5**: games that combine a high complexity (3.07), with the highest possible popularity; represent games that have become cult phenomena.
- **Cluster 0**: it includes hardcore-heavy games, represented by the maximum **ComWeight**, max popularity and the highest **ComMaxPlatime**.

Table 4: Distribution of categories across clusters (Absolute vs Relative %)

Cluster	War		Strategy		Family		Thematic		Abstract		Childrens		Party		CGS	
	abs %	rel %	abs %	rel %	abs %	rel %	abs %	rel %	abs %	rel %	abs %	rel %	abs %	rel %	abs %	rel %
0	65.62	86.69	10.15	8.27	0.44	0.34	8.55	3.42	1.00	0.38	0.00	0.00	0.49	0.11	6.96	0.79
1	1.33	3.68	0.39	0.67	12.30	20.00	2.50	2.09	30.37	24.41	59.41	39.83	13.19	5.99	14.11	3.34
2	15.66	37.48	13.98	20.65	7.22	10.17	14.59	10.58	14.82	10.31	0.86	0.50	9.31	3.66	32.48	6.65
3	1.77	2.37	13.99	11.57	64.21	50.66	16.91	6.87	20.76	8.09	14.84	4.83	65.25	14.37	10.72	1.23
4	7.26	21.98	2.50	4.68	7.99	14.23	7.12	6.53	29.95	26.36	24.88	18.27	8.82	4.39	13.73	3.56
5	8.37	12.33	58.98	53.60	7.85	6.80	50.32	22.45	3.10	1.33	0.00	0.00	2.94	0.71	22.01	2.77

The table indicates, for each game category, the relative percentage (rel) that is the fraction of the cluster composed of that category, while the absolute percentage (abs) that is the fraction of the total occurrences of that category present in each cluster. Note that games that appeared in multiple categories have been divided into percentages for each category.
The most obvious analysis concerns Cluster 0 and Cluster 1, which respectively confirm their composition:

- Cluster 0 reflects high complexity and recommended age, as it holds a 'monopoly' on war games.
- Cluster 1 reflects the reputation of party games, as most card and family games are found in this group.

4 Classification

For the classification process the target attribute is **Rating**.
the dataset was divided into 2 smaller subsets, one for training (60%) and one for testing (40%), a stratification was applied to ensure that the class distribution remained consistent across both sets. The following features were engineered to improve the classification algorithm performance:

- PlayerRange = MaxPlayers - MinPlayers
- PassedYears = 2026 - YearPublished
- WeightPerAge = ComWeight/(ComAgeRec +1)

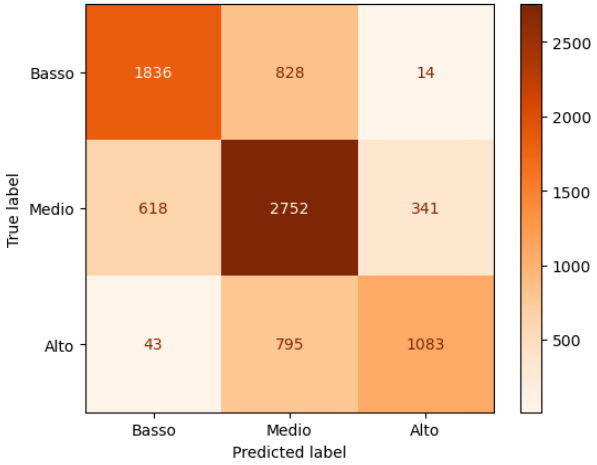
However, after preliminary experiments, only PassedYears was retained among the engineered features, as the other two did not provide performance improvements and in some cases negatively affected the models.
The performance of all algorithms was evaluated with the following metrics:accuracy, ROC curve,confusion matrix, F1 score. To find the optimal parameters a grid search with stratified cross validation was used.

4.1 KNN

First, the **StandardScaler()** function was applied to the data to standardize all numerical features and prevent variables with larger scales from dominating the distance computations. The scaler was fitted on the training set and subsequently applied to both the training and test sets in order to avoid data leakage. To determine the optimal value of number of neighbor, a Grid Search was performed over the candidate values [80, 90, 100, 110, 120]. Each configuration was evaluated using stratified 5-fold cross-validation, where the training data was split into five stratified subsets. In each iteration, four folds were used for training and one for validation, ensuring that the class distribution was preserved across all folds. The final performance of each candidate value was computed as the average accuracy across the five validation folds, and the configuration achieving the best mean score was $n_neighbors = 100$. Several different feature combinations were evaluated during the training and testing phases. The combination that achieved the best performance is the following: [**"PassedYears", "ComWeight", "belong_to_family", "NumOwned", "NumWish"**].
The tables and figures below present the main performance metrics and the correlation matrix.

Class	Precision	Recall	F1-Score	Support
1 – Low	0.74	0.69	0.71	2,678
2 – Medium	0.63	0.74	0.68	3,711
3 – High	0.75	0.56	0.64	1,921
Accuracy		0.68		8,310
Macro Avg	0.71	0.66	0.68	8,310
Weighted Avg	0.69	0.68	0.68	8,310

(a) Metrics



(b) Confusion Matrix

Figure 14: KNN Model Performance: Metrics and Confusion Matrix.

The KNN algorithm shows high proficiency in classifying the extreme classes ('Low' and 'High'), achieving AUC scores of 0.89 and 0.90. However, performance degrades for the 'Medium' class (AUC 0.77). Being an intermediate class, 'Medium' is packed in between , leading to higher neighborhood impurity in transition zones. Furthermore, the class imbalance (high support) introduces a bias in the KNN majority voting mechanism, causing the model to over-predict the 'Medium' label in uncertain regions, as evidenced by the low Precision (0.63).

4.2 Naive Bayes

The Naive Bayes classifier works under the assumption that all the variables are independent from each other given the class label. Although this assumption is often unrealistic, the model still performs well in many real-world classification tasks; for the CategoricalNB a new feature was engineered : `PlayTime = ComMaxPlaytime - ComMinPlaytime`]
The same set of attributes used for the KNN model was also employed for Naive Bayes, with the `GaussianNB` implementation being applied, standardization and normalization were not necessary because this algorithm does not rely on distances between points, but rather on probability distributions. To tune

the **var_smoothing** parameter (which adds a small value to the variance of each feature), a stratified 5-fold cross-validation was performed. The best performance was achieved with `var_smoothing = 1.0`, the maximum possible value for this parameter. This result indicates that the algorithm's assumptions are quite far from the actual data distribution, which is consistent with the observed skewness of the features, The **priors** parameter was left at its default value, allowing the algorithm to automatically compute class probabilities based on their relative frequencies in the training set. As anticipated, the algorithm showed lower performance compared to KNN: from the Confusion Matrix, we can observe a tendency of the model to misclassify 'Medio' instances as 'Basso', which significantly impacts the recall of Class 2 also the model struggles particularly with Class 3 (Recall 0.52), often confusing it with Class 2.

	Prec.	Recall	F1	Supp.
Class 1	0.62	0.73	0.67	2678
Class 2	0.59	0.56	0.57	3711
Class 3	0.62	0.52	0.57	1921
Accuracy			0.61	8310
Macro Avg	0.61	0.60	0.60	8310
Weighted Avg	0.61	0.61	0.60	8310

Table 5: Classification Report

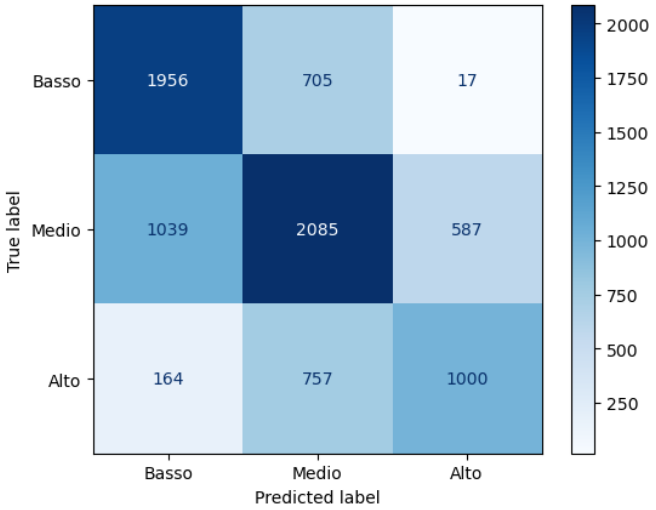


Figure 15: Confusion Matrix

The **CategoricalNB** implementation was tested in order to achieve better performance, both using the same set of features adopted for GaussianNB and using a dedicated subset composed exclusively of strictly categorical variables["BelongToFamily","PlayerRange", "Kickstarted","IsReimplementation","NumImplementations"]. However, both approaches yielded poor results in terms of accuracy respectively (51% and 41%), which led to no further investigation.

4.3 Decision Tree

For the Decision Tree the best features were **["PassedYears", "ComWeight", "ComAgeRec", "NumOwned", "NumWish"]**: the same set used for KNN and Naive Bayes, with ComAgeRec in place of belong_to_family. The initial model was trained using the default hyperparameters. This resulted in a training accuracy of 99% and a test accuracy of 59%, clearly indicating a strong overfitting problem. Hyperparameter tuning was then performed, focusing on three key parameters: *criterion* ([`gini`, `entropy`]), *max_depth* ([3, 5, 7, 10, 12, 14, 16]), and *min_samples_leaf* ([10, 20, 50, 100]). A Grid Search with 5-fold cross validation on the training set was used to identify the optimal combination of these parameters. The best configuration obtained was the following:

- criterion = Gini
- max_depth = 12
- min_sample_leaf = 50

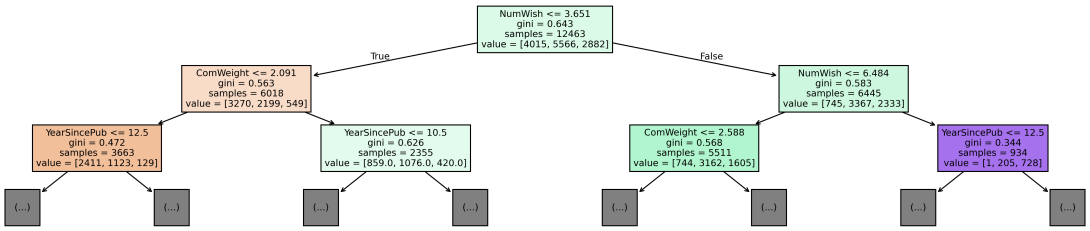


Figure 16: Visualization of the decision tree up to depth 3

The decision tree identifies **NumWish** as the primary splitting attribute at the root, followed by **ComWeight** and **PassedYears** at the subsequent levels.
As observed, the initial impurity at the root node is quite high,considering that the maximum Gini index is 0.66 for a three-class problem. However, the impurity consistently decreases as the tree grows from the root down to the leaves, demonstrating the model's ability to progressively separate the classes. .

	Prec.	Recall	F1	Supp.
Class 1	0.71	0.71	0.71	2678
Class 2	0.63	0.70	0.66	3711
Class 3	0.72	0.58	0.64	1921
Accuracy			0.67	8310
Macro Avg	0.69	0.66	0.67	8310
Weighted Avg	0.68	0.67	0.67	8310

Table 6: Classification Report

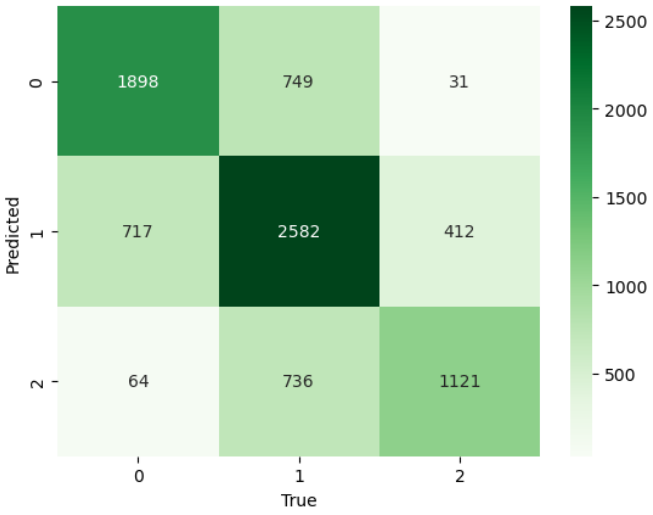


Figure 17: Confusion Matrix

The overall accuracy of the model on the test set is 67%, class 1 ("Low") is the most stable class, showing both high precision and high recall. Class 2 ("Medium") is characterized by low precision, indicating a higher rate of false positives for this class. In contrast, Class 3 ("High") exhibits low recall, meaning that the model struggles to correctly identify instances belonging to this category.

4.4 Final discussion

Before comparing the classification models, it is useful to examine the distribution of the data points by Rating, using PCA.

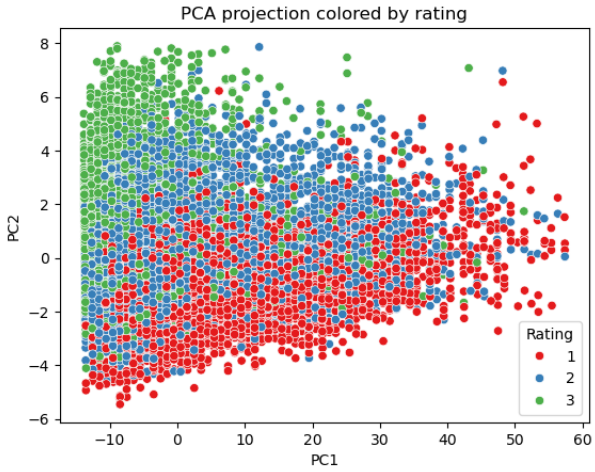
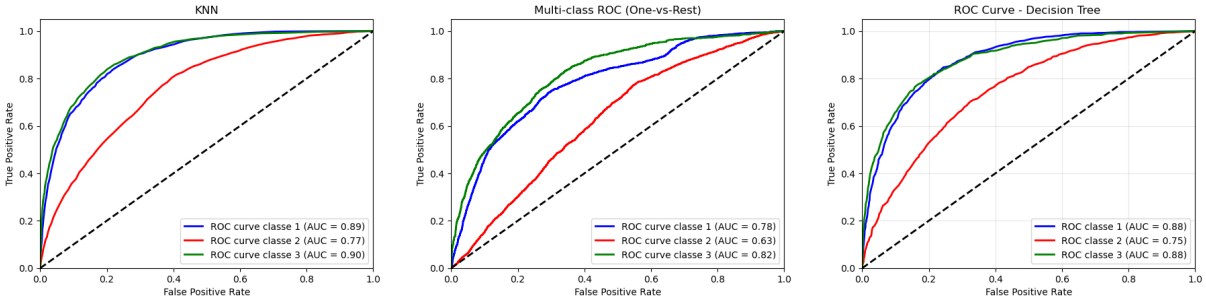


Figure 18: The features used for the PCA are the same used in every classifier

The PCA projection clearly shows a strong overlap among the classes, making it difficult to identify well-defined cluster structures, especially for Class 2 ("Medium"), this helps explain the lower performance observed for this class across all the classifiers used.
It is important to note that the observed class overlap does not necessarily indicate poor data quality, game ratings are inherently subjective, as they reflect individual user preferences rather than objective measurements. As a result, games with very different characteristics may receive similar ratings, naturally leading to overlapping feature patterns across classes. Therefore, the difficulty in class separation is likely due to the intrinsic nature of the target variable rather than noise or errors in the dataset.



(a) KNN **(b)** Naive Bayes **(c)** Decision Tree

Figure 19

The ROC curve analysis shows that Naive Bayes is the least performant model in terms of class discrimination (macro ROC-AUC 0.74), while KNN achieves the highest ROC-AUC (0.85), performing slightly better than the Decision Tree (0.84). The same ranking holds for threshold-dependent metrics: KNN reaches an accuracy and a macro F1-score of 0.68, the Decision Tree 0.67 and Naive Bayes 0.61 and 0.60. KNN is therefore the best classifier overall, both in ranking the instances across all thresholds and in the final discrete classification.

The Decision Tree remains very close to KNN: the regularization parameters selected by the grid search (max_depth = 12, min_samples_leaf = 50) reduce overfitting and make the tree robust to noisy patterns, bringing its performance near that of KNN while keeping a much more interpretable model. Naive Bayes is penalized by its independence assumption, since the selected features are correlated with each other.

5 Regression

In this part the dataset has been analyzed using different regression models, where the first task in which a univariate regression has been accomplished, evaluating the target variable *NumWish* with *NumOwned* as a primary proxy for social proof and established market presence; in this part a **Linear Regression**, **Ridge** and **Lasso** methods have been used. The model's been subsequently expanded into a multivariate analysis, incorporating *ComWeight* and *YearPublished*, to account for the complexity and target audience of the title, and YearPublished, to adjust for temporal trends and modern industry 'hype'; in this case also Non Linear models such as **KNN** or **Decision Tree Regressor** have been used.

5.1 Univariate Regression

Metric	Linear	Ridge	Lasso
R^2	0.647	0.647	0.443
MSE	0.925	0.925	1.460
MAE	0.765	0.765	0.975

Table 7: Regression Models Performance (Generalization Error)

The univariate analysis confirms that NumOwned is a powerful predictor of game desirability. The baseline linear model achieves an R^2 of 0.647, explaining nearly 65% of the variance with a single feature. This result is perfectly consistent with the previously observed Pearson correlation of 0.80. While Ridge regression yields identical results, the standard Lasso model shows a significant performance drop ($R^2 = 0.443$), indicating that its default penalty is too aggressive for a single-variable setup. Furthermore, the minimal gap between the Training Loss and the Generalization Error demonstrates the model's high stability and absence of overfitting.

5.2 Multivariate Regression

Metric	Linear	Ridge	Lasso	Dec Tree	KNN
R^2	0.745	0.745	0.493	0.765	0.782
MSE	0.668	0.668	1.328	0.615	0.572
MAE	0.645	0.645	0.919	0.616	0.594

Table 8: Generalization Error Comparison (Test Set)

The multivariate analysis incorporates NumOwned, ComWeight, and YearPublished. To identify the optimal configuration for each non-linear algorithm, **Grid Search** has been implemented in conjuction with **5 fold Cross-Validation** on the training data. Regarding linear models, Linear and Ridge regressions provide a solid baseline ($R^2 = 0.745$), while the standard Lasso model underperforms significantly ($R^2 = 0.493$). This is due to the default penalty ($\alpha = 1.0$) being too aggressive, causing the model to zero out relevant features (underfitting). However, by implementing Lasso CV to optimize the shrinkage parameter, the model recovers its predictive power, reaching an R^2 of 0.745. This highlight the necessity of hyperparameter optimization in regularized models.
For the non-linear models, **Grid Search** identified the best hyperparameters for the **K-NN regressor** as $k = 50$ *neighbors*, the *cityblock (Manhattan)* distance metric and *uniform* weights. Similarly, the best configuration for **Decision Tree Regressor** involved a *max_depth* of 21,*min_samples_split* of 2, and *min_samples_leaf* of 32, using *squared_error* as the criterion.

Final discussion The KNN Regressor stands out as the most effective model. The performance gain observed with the R^2, improving from 0.745 (linear baseline) to 0.782, confirms that the factors driving player interest follow complex, non-linear patterns that a global linear trend cannot fully capture.The model demonstrates high predictive stability, ensuring reliable generalizations on unseen data.

6 Pattern mining

To perform pattern mining we needed a little pre-processing in order to transform all numerical features into categorical, so we dropped **BGGId** and **Name** because are not useful to the analysis, then **NumWeightVotes** and **NumWish** because these features were going to create trivial association rules with **NumOwned**, and finally **MfgAgeRec**, that has the same purpose as **ComAgerec**. Then we performed the sequent transformations:

- **YearPublished**: classic(<=2000), modern(2001-2015), recent(>=2016).
- **ComWeight**: easy, medium_diff, difficult.
- **NumOwned**: low_popularity, medium_popularity, popular, best_seller.
- **rating**: low_rating, medium_rating, high_rating.
- Made the average between **MinPlayers** and **MaxPlayers** to obtain the column **NumPlayers**: solo/couple_games (1-2), group_games(2-4), party_games(5+). This was done becaues of the high probability of having a lot of itemsets with these two columns together (ex. low **MaxPlayers** with low **MinPlayers** and viceversa)
- Same thing did on **ComMinPlaytime** and **ComMaxPlaytime** to obtain the column **Playtime**: short_time, medium_time, long_time, very_long_time.
- **ComAgeRec**: kids(9-), pre_teens(10-13), teens/adults(14+).
- **LanguageEase**: easy_language, medium_language, difficult_language.
- **Others**: The column **NumAlternates** had a very high percentage of zeros (>77%), so was divided into Has_alternates, Has_not_alternates, same for **NumExpansions** and **NumImplementations**. Binary columns were transformed into yes or no (ex. **Cat:War** became Cat:war and No_Cat:War). Then the values starting with **"No"** or Then the values starting with "No" or "has_not" were removed, because these created a lot of redundant itemsets due to the fact that a lot of game have not a certain category and have not alternates, implementations or expansions.Then, values starting with **No** or **Has_not** were removed, as they generated a large number of redundant itemsets and

rules due to the fact that a lot of game do not belong to a certain category and do not hahve alternates, implementations or expansions.

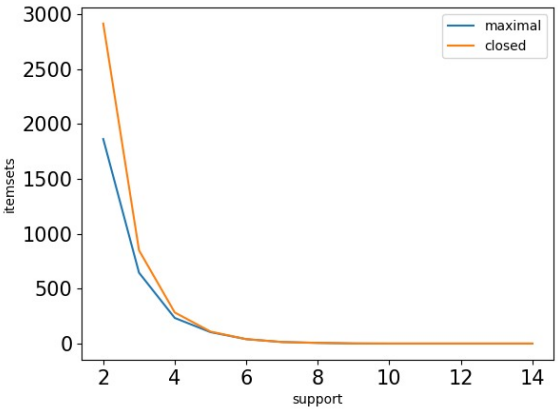


Figure 20: Support plot

We used the support plot to decide the best support percentage to use, and we decide to use 4%, with a $z_{min} = 4$, because by using lower z_{min} the rules found were trivial and obvious, with a $z_{min} = 4$ we found some interesting rules. As we can see by using the **elbow method**, after support = 4 maximal and close itemsets began to be coincident.
After that we calculated, by using the apriori algorithm, the frequent, closed and maximal itemsets.

As we can see in the table below, our dataset has a lot of kids games, meant for groups and pretty easy to play, usually with a short duration time, low ratings and with alternates available. We didn't insert frequent itemsets in the table below because they are coincident with closed itemsets.

Itemset Type	Itemset	Support
Closed	kids, easy, has_alternates, group_games	9.69
	kids, easy, short_time, group_games	8.97
	low_rating, easy, modern, group_games	8.57
Maximal	medium_diff, medium_rating, pre-teens, group_games	7.86
	medium_diff, modern, pre-teens, group_games	6.99
	medium_rating, modern, pre-teens, group_games	6.89

Table 9: Frequent, closed and maximal itemsets with support

By running the FP-growth subsequently, we obtained the same results and the same itemsets, so there is no need to add also the table obtained by the FP-growth, because that would have been redundant.

6.1 Association rules

We took the rules with a confidence above 70 and ordered them by lift, we decided to not take the first rules with higher lift because they had all the same consequent and were redundant, so we took 6 interesting rules with 6 different consequent to analyze rules for different features. The rules are listed in the table below, the first rule states that if a game has a very long duration time, has a minimum age of 14 and is for 1 or 2 players, then the 93% of the time it is a war game, at the same time it has a lift of 5.983, that means that a game with these antecedent feature has a probability to be of war category that is almost 6 times higher than the probability to be **cat:War** of a game randomly chosen in the dataset. With the same method we can read the other rules.

Antecedent	Consequent	Abs. Supp.	Supp. (%)	Conf.	Lift
teens/adults (14+), very_long_time, solo/couple_games	Cat:War	1385	6.667	0.939	5.983
very_long_time, solo/couple_games, medium_popularity	teens/adults (14+)	1019	4.905	0.708	4.739
teens/adults (14+), Cat:War, medium_popularity	very_long_time	1200	5.777	0.834	3.442
teens/adults (14+), Cat:War, medium_popularity	solo_couple_games	1184	5.700	0.790	2.872
classic (<2000), easy, group_games (2-4 players)	low_rating	1020	4.910	0.789	2.450
short_time, easy, Has_Alternates, group_games	kids ($\leq$9)	1189	5.723	0.740	2.240

Table 10: Most interesting association rules extracted from the dataset